STANFORD DEPARTMENT OF ECONOMICS

Economics Social Calendar

User and maintainer guide

1. What this is

A shared web calendar for the Department of Economics. Anyone with the link can see the social events happening in the department — seminars, receptions, happy hours, visiting speakers — without logging in. People can subscribe to it from their own calendar app, so events appear alongside their normal schedule and update on their own.

Adding, changing and removing events requires a passphrase. Only a handful of people need it.

All times are shown in Pacific Time no matter where the reader is.

2. Using the calendar

2.1 Finding things

Switch between **Year**, **Month**, **Week** and **Day** with the buttons at the top right. The year view is laid out like a printed wall planner, with months across and days down, which is the quickest way to see how a term is filling up. Clicking any day opens it.

Keyboard shortcuts: ← and → move back and forward, `T` jumps to today, and `Y` `M` `W` `D` switch views. The search box filters by title, location, details or contact.

2.2 Tags

Every event has a colour-coded tag. Click a tag in the sidebar to hide it, and click again to bring it back. A tick in the box means the tag is showing. Everything else on the page follows the filter, including the exports.

2.3 Getting events into your own calendar

Three options, in the sidebar:

- **Subscribe** to a live feed. Your calendar app re-checks the address on its own, so new and changed events arrive without you doing anything. Choose the whole calendar or a single tag. This is the option most people want.
- **Download** an `.ics` file of whatever is on screen. Filter to one tag first and you get only that tag. This is a one-time copy: it will not update later.
- **Add to Google Calendar** from inside a single event, for when you only want that one thing.

2.4 Printing

The Export panel produces a PDF of the view you are looking at — a wall planner for a year, a grid for a month or week, an agenda for a day — followed by the full details of every event in that period. Whatever you have filtered out stays out of the PDF.

3. Adding and changing events

Click **Log in to edit** at the top right and enter the passphrase. You stay logged in for twelve hours, or until you click **Log out**.

Once logged in you can:

- **Add event** — the button at the top of the sidebar.
- **Edit** or **Remove** — open any event and use the buttons at the bottom. Removing asks for confirmation.
- **Manage tags** — rename them, change colours, add new ones or remove them. Removing a tag does not delete its events; they simply become untagged.
- **Rename the calendar** is not done here — see section 6.

3.1 Event fields

Title and the start and end times are required. Everything else is optional but worth filling in: location, point of contact and their email, details, and a link. The contact details appear in the event and travel with it into people's own calendars, so attendees know who to ask.

Tick **All-day event** for something without a specific time, such as a deadline or a multi-day visit.

3.2 Drafts

The event form has two save buttons. **Add and publish** makes the event visible to everyone. **Save as draft** keeps it visible only to people logged in to edit — it stays out of the feed, the downloads and the PDF.

Use a draft when a date or venue is not confirmed. Drafts appear hatched and labelled in the calendar so they are hard to confuse with real events. Publish one later by opening it and clicking **Publish**.

3.3 If two people edit at once

If someone else saved a change while you had the same event open, your save is refused and their version is loaded instead, with a message explaining what happened. Make your change again on top of theirs. Nothing is lost silently.

4. How it is put together

Four services, all free, currently registered under the maintainer's personal account.

Service	Role	Notes
GitHub	Holds the code	Render deploys automatically whenever the main branch changes.
Render	Runs the website	Free tier. Sleeps after 15 minutes of no traffic and takes up to a minute to wake.
Turso	Stores the events	A hosted SQLite database. Nothing is stored on Render itself, so deploys never affect the data.
UptimeRobot	Keeps it awake	Requests the site every 10 minutes so it does not sleep, and emails if the site goes down.

The separation that matters: **the events live in Turso, not on Render**. Redeploying, restarting or rebuilding the website cannot lose them.

5. Handing this over

If maintenance moves to someone else, they will need their own accounts. The data can be copied across; nothing is tied to the original owner except the accounts themselves.

1. **Take the code.** Fork or clone the GitHub repository into an account the new maintainer controls.

2. **Create a database on Turso.** Sign up, install the CLI, then:

   ```
   turso db create econ-calendar
   turso db show econ-calendar --url
   turso db tokens create econ-calendar
   ```

   Keep the URL and the token. To carry the existing events across, ask the outgoing maintainer for a dump (`turso db shell <old-db> ".dump" > backup.sql`) and load it into the new database.

3. **Create the web service on Render.** Connect the repository and set:

   Build command: `npm ci && npm run build`
   Start command: `npm start`
   Health check path: `/healthz`

   Then add exactly three environment variables:

   `TURSO_DATABASE_URL` — from step 2
   `TURSO_AUTH_TOKEN` — from step 2
   `ADMIN_PASSPHRASE` — the editing passphrase, at least 8 characters

   The service refuses to start without the first of these, rather than quietly running against a database that would be wiped on the next deploy. Changing `ADMIN_PASSPHRASE` later rotates the passphrase and logs everyone out.

4. **Set up UptimeRobot.** Add an HTTP monitor pointing at `https://<your-site>/healthz`, checking every 10 minutes, with an alert to the new maintainer's email. Without this the site sleeps and the first visitor each morning waits about a minute.

5. **Check it worked.** Open `/healthz` in a browser. It should report:

   `"remote": true` and `"persistent": true`

   Then log in, add a test event, redeploy, and confirm the event is still there. That last step is the one that actually proves the setup is sound.

5.1 Keeping an eye on the free tiers

None of the four services costs anything at the department's scale, but the limits are worth knowing:

- **Render** allows 750 instance-hours a month per workspace. Keeping one site awake around the clock uses almost all of it, so avoid running a second free service in the same workspace.
- **Turso** allows 500 million database rows read a month. Normal departmental use is a small fraction of that.
- **Free tiers change.** If the site starts sleeping or refusing to load, check whether a limit was reached before assuming something broke.

Paying roughly $7 a month for Render removes the sleeping behaviour entirely and makes UptimeRobot optional.

6. Making changes to the calendar itself

A few things are set in the code rather than in the website, because they change once a year at most. Edit the file, commit, and Render redeploys.

To change	Edit
The calendar's name	`ORG_NAME` in `src/config.js`
The academic year in the header	`CUR_YEAR` in `src/config.js`
The timezone	`CALENDAR_TZ` in `src/lib/tz.js`
The icon in the browser tab	`public/favicon.svg`

7. Backups

The events live in one hosted database. To take a copy:

```
turso db shell econ-calendar ".dump" > backup-$(date +%F).sql
```

Worth doing once a term, and before any handover. A rougher second copy is available by logging in and downloading the `.ics` file, which contains every event including drafts in a format any calendar can read.

8. Known limits

- **No repeating events.** A weekly seminar has to be entered as separate events.
- **One shared passphrase.** Everyone who can edit uses the same one, and the calendar does not record who changed what. Share it narrowly.
- **A feed link containing a token shows drafts.** Calendar apps store such links in plain text, so treat one as a password rather than something to paste into a group chat.

Source code and technical documentation are in the repository README.